

CheckFlow

Sistema de Gestão de Checklists

Documentação Técnica e Funcional

Versão 1.0 · 05/06/2026

Produto desenvolvido com React · TypeScript · Supabase · Railway

1. Visão Geral do Produto

O **CheckFlow** é um sistema web de gestão de checklists operacionais, projetado para empresas que precisam padronizar, distribuir e acompanhar a execução de atividades de campo em tempo real. O produto oferece um construtor visual de checklists com seções, atividades tipadas e lógica condicional, além de integração com WhatsApp via Evolution API para notificações e recuperação de senha.

Ambientes do sistema

Ambiente	Público-alvo	Função principal
/gestao	Gestor / Analista	Configurar checklists, usuários, grupos, documentos e catálogos
/operacao	Operador de campo	Executar e preencher checklists em tempo real
/sistema	Admin SaaS	Gerenciar empresas, WhatsApp e configurações globais

2. Stack Tecnológico

Camada	Tecnologia	Versão / Detalhe
Frontend	Next.js + React	v16.2 / React 19.2 – App Router
Linguagem	TypeScript	Strict mode, sem any
Estilização	Tailwind CSS	v4 – utilitários, sem CSS modules
Backend/DB	Supabase	Postgres + Auth + Storage + RLS
API Server	Fastify	v5.8 – porta 3001, CORS + Helmet
Hosting	Railway	Deploy via railway up
Mensageria	Evolution API	WhatsApp Web – instância checkflow
Ícones	Lucide React	v1.17
Package manager	npm	Monorepo com workspaces

3. Arquitetura e Estrutura de Pastas

O projeto é organizado como um **monorepo npm** com dois apps e um pacote de tipos compartilhados:

```
checkFlow/  
  ■■■ apps/  
    ■ ■■■ web/ # Next.js (frontend + API routes)  
    ■ ■ ■■■ app/ # App Router - páginas por ambiente  
    ■ ■ ■■■ components/ # Componentes reutilizáveis  
    ■ ■ ■■■ contexts/ # SessionContext global  
    ■ ■ ■■■ lib/ # Supabase clients (browser + server)  
    ■ ■■■ api/ # Fastify (integração WhatsApp, sync usuários)
```

```
■ ■■■ src/routes/ # health, usuarios, catalogos, whatsapp
■■■ packages/
■ ■■■ types/ # Tipos TypeScript compartilhados
■■■ supabase/
■■■ migrations/ # 17 arquivos .sql versionados
```

Padrões de código

- Componentes: PascalCase · Hooks/utils: camelCase · Rotas: kebab-case
- Imports via alias @/ (absolute paths)
- Estado servidor: React Query · Estado UI: Zustand
- Formulários: React Hook Form + Zod
- Tipos reutilizados exportados de src/types/

4. Banco de Dados – Supabase / Postgres

Todas as alterações de schema são versionadas em **supabase/migrations/** com timestamp. RLS está habilitado em todas as tabelas por padrão. PKs utilizam UUID com `gen_random_uuid()`.

4.1 Regras absolutas

- PKs: UUID com `gen_random_uuid()`
- Nomes de colunas: `snake_case`
- RLS: obrigatório em toda tabela com dados de usuário
- SQL bruto nunca no frontend – sempre Supabase client ou Edge Function
- Migrations: `YYYYMMDDHHMMSS_descricao.sql`

4.2 Tabelas principais

Tabela	Descrição
empresas	Empresas clientes (status: pending / active / inactive / blocked)
unidades	Unidades/filiais de cada empresa
grupos	Grupos com label configurável por unidade (Grupo / Setor / Departamento)
subgrupos	Subgrupos dentro dos grupos
usuarios	Perfis de usuário vinculados ao Supabase Auth
perfis	Papéis: admin_sistema, admin_empresa, operacao
permissoes	Permissões granulares (recurso + ação)
perfil_permissoes	Vínculo papel ↔ permissão
usuario_empresa	Relacionamento usuário ↔ empresa com papel
usuario_unidade	Acesso do usuário a unidades específicas
sessao_usuario	Última sessão (ambiente + empresa + unidade) para restauração

4.3 Módulo de Checklists

Tabela	Descrição
checklists	Template com versionamento (rascunho / publicado / inativo)
checklist_versoes	Snapshot imutável de cada publicação (JSON completo)
checklist_secoes	Seções/grupos dentro do checklist com ordenação
checklist_atividades	Atividades tipadas com config JSONB e flags
checklist_atividade_opcoes	Opções de múltipla escolha com validade

4.4 Módulo de Documentos e Configuração

Tabela	Descrição
documentos	POP, IT e Consultas Inteligentes com etapas e vídeos
documento_etapas	Passos com título, conteúdo e vídeo YouTube
etapa_imagens	Imagens em carrossel por etapa
catalogos	Valores de catálogo (dropdowns dinâmicos)
causa_raiz	Motivos de análise de causa raiz
nao_execucao	Motivos de não execução com tipos
empresa_importacao_api	Config de API externa para sync de usuários
usuario_fonte_sync	Rastreamento de sincronização por usuário

4.5 Migrations criadas (17 arquivos)

Timestamp	Arquivo	Conteúdo
20260603000001	foundation.sql	Schema base: enums, empresas, unidades, usuarios, perfis, permissoes, sessao_us
20260603000002	policies.sql	Políticas RLS gerais
20260603000003	admin_policies.sql	Políticas RLS para admin_sistema
20260603000004	storage.sql	Buckets de armazenamento
20260603000005	seed_unidades.sql	Dados de seed para unidades
20260603000006	grupo_labels.sql	Labels customizáveis para grupos/subgrupos
20260603000007	formatacao.sql	Configuração de formatação de UI
20260603000008	documentos.sql	Documentos POP/IT com etapas e imagens
20260603000009	documento_arquivo.sql	Anexos de documentos
20260603000010	nao_execucao.sql	Motivos de não execução
20260603000011	nao_execucao_tipo.sql	Tipos de motivos de não execução
20260603000012	causa_raiz.sql	Análise de causa raiz
20260603000013	catalogos.sql	Catálogos de valores
20260603000014	catalogo_api.sql	Integração de catálogo via API
20260603000015	empresa_importacao.sql	Config de importação de usuários por empresa
20260603000016	usuario_fonte_sync.sql	Rastreamento de sync de usuários
20260603000017	checklists.sql	Módulo completo de checklists: templates, seções, atividades, opções, versões

5. Módulos Funcionais Implementados

5.1 Autenticação e Sessão

O login aceita **CPF** ou **e-mail**. Se CPF, o sistema resolve o e-mail correspondente na tabela usuarios. Após autenticação via Supabase Auth, o papel do usuário é verificado e a última sessão (ambiente + empresa + unidade) é restaurada automaticamente de sessao_usuario.

- Formatação automática de CPF (XXX.XXX.XXX-XX)
- Opção 'Manter conectado'
- Recuperação de senha por e-mail e via WhatsApp
- Middleware Next.js protege todas as rotas autenticadas
- admin_sistema é redirecionado para /sistema; demais para /gestao

5.2 Contexto de Sessão Global (SessionContext)

Gerencia o estado global compartilhado entre todos os componentes do app:

- ambiente ativo: gestao / operacao / sistema
- empresaAtiva e unidadeAtiva – seleção persistida no DB
- Lista de empresas e unidades acessíveis ao usuário
- modoEmpresa – admin visualizando empresa específica
- grupoLabel / subgrupoLabel – terminologia customizável por unidade (ex: Setor, Área, Loja)

5.3 Construtor de Checklists (Montador)

O construtor visual permite criar checklists com estrutura hierárquica: **Checklist** → **Seções** → **Atividades** → **Dependências condicionais**.

Tipos de atividade disponíveis:

Tipo	Descrição	Config especial
sim_nao	Resposta Sim/Não	Pode disparar atividades filhas
numero	Valor numérico	Min, max, unidade de medida
texto	Campo de texto livre	Suporte a leitura QR/código de barras
multipla_escolha	Seleção de opções	Opções com validade; dispara dependentes
catalogo	Dropdown de catálogo	Vinculado à tabela catalogos
foto	Captura de imagem	Câmera nativa
assinatura	Assinatura digital	Canvas touch/mouse
data_hora	Data e hora	Preenchimento automático
localizacao	GPS	Raio de tolerância configurável

Flags de atividade:

- Obrigatória – impedida de salvar sem resposta (padrão ativado)
- Crítica – reprovação da atividade reprova o checklist inteiro
- Gera plano de ação – abre workflow de ação corretiva em caso de falha

Versionamento:

- Checklists podem ser salvos como Rascunho sem publicar
- Publicar cria um snapshot imutável em `checklist_versoes` (JSON completo)
- Versões são numeradas sequencialmente e nunca mudadas
- Checklists publicados podem ser inativados, mas não editados retroativamente

5.4 Gestão de Acesso

- Empresas → Unidades → Grupos → Subgrupos (hierarquia completa)
- Usuários podem ter perfis distintos por empresa
- Perfis definem permissões granulares (recurso + ação)
- Importação em massa de usuários via API externa com mapeamento de campos
- Sync automático: cria, atualiza e inativa usuários conforme a fonte

5.5 Configurações do Sistema

- Catálogos – valores dinâmicos para dropdowns de atividades
- Documentos (POP/IT/Consulta Inteligente) com etapas, conteúdo rico e vídeos YouTube
- Motivos de não execução e tipos
- Análise de causa raiz
- Labels customizáveis (Grupo → Setor, Subgrupo → Área/Loja)

5.6 Integração WhatsApp (Evolution API)

Integração com **Evolution API** hospedada no Railway para envio de mensagens via WhatsApp Web autenticado por QR Code.

- Página `/sistema/whatsapp`: status de conexão em tempo real (polling 5s)
- Geração de QR Code para vincular número
- Envio de mensagens de texto via `POST /whatsapp/enviar`
- Recuperação de senha enviada diretamente via WhatsApp
- Instância: `checkflow` | URL: `evolution-api-production-d484.up.railway.app`

6. Mapa de Rotas da Aplicação

Rota	Arquivo	Função
/	app/page.tsx	Redireciona para /login
/login	(auth)/login/page.tsx	Login por CPF ou e-mail
/recuperar-senha	(auth)/recuperar-senha/page.tsx	Solicitar reset de senha
/nova-senha	(auth)/nova-senha/page.tsx	Definir nova senha
/gestao	gestao/page.tsx	Home do ambiente de gestão
/gestao/checklists	gestao/checklists/page.tsx	Listagem de checklists
/gestao/checklists/novo	gestao/checklists/novo/page.tsx	Criar checklist
/gestao/checklists/[id]	gestao/checklists/[id]/page.tsx	Visualizar checklist
/gestao/checklists/[id]/montar	gestao/checklists/[id]/montar/page.tsx	Construtor visual
/gestao/acessos/empresa	gestao/acessos/empresa/page.tsx	Unidades da empresa
/gestao/acessos/usuarios	gestao/acessos/usuarios/page.tsx	Gerenciar usuários
/gestao/acessos/perfis	gestao/acessos/perfis/page.tsx	Perfis e permissões
/gestao/grupos	gestao/grupos/page.tsx	Grupos
/gestao/grupos/[id]/subgrupos	gestao/grupos/[id]/subgrupos/page.tsx	Subgrupos
/gestao/configuracoes/catalogos	gestao/configuracoes/catalogos/page.tsx	Catálogos
/gestao/configuracoes/documentos	gestao/configuracoes/documentos/page.tsx	POP / IT
/gestao/configuracoes/causa-raiz	gestao/configuracoes/causa-raiz/page.tsx	Causa raiz
/gestao/configuracoes/nao-execucao	gestao/configuracoes/nao-execucao/page.tsx	Não execução
/operacao	operacao/page.tsx	Home do operador de campo
/sistema	sistema/page.tsx	Home admin SaaS
/sistema/empresas/[id]	sistema/empresas/[id]/page.tsx	Gerenciar empresa
/sistema/whatsapp	sistema/whatsapp/page.tsx	Status e QR WhatsApp

7. API Server (Fastify)

Servidor Fastify rodando na porta **3001** com CORS habilitado e headers de segurança via Helmet. Responsável por operações que requerem service-role do Supabase ou comunicação com APIs externas (WhatsApp, sync de usuários).

Método	Rota	Descrição
GET	/health	Health check – retorna status do servidor
GET	/whatsapp/status	Verifica estado da conexão WhatsApp
POST	/whatsapp/conectar	Cria instância e retorna QR Code
POST	/whatsapp/enviar	Envia mensagem de texto
POST	/whatsapp/recuperar-senha	Envia link de recuperação via WhatsApp
POST	/usuarios/sync-all	Sincroniza usuários de APIs externas por empresa
GET	/catalogos	Lista catálogos disponíveis

Lógica de sync de usuários

- Lê configuração de importação por empresa (URL, headers, mapeamento de campos)
- Busca usuários na API externa
- Cria no Supabase Auth + tabela usuarios se não existir
- Atualiza dados de usuários existentes
- Inativa usuários removidos da fonte (estratégia: inativar)
- Registra fonte em usuario_fonte_sync (nome da API, sistema)
- Atribui papel 'Operação' na empresa automaticamente

8. Skills Claude Code (.claude/skills/)

O projeto conta com 7 skills configuradas para o Claude Code que garantem consistência arquitetural, economiam tokens e evoluem junto com o projeto.

Skill	Comando	Função
arch	/arch	Regras de stack e convenções de código
biz	/biz	Regras de negócio e fluxos do produto
uimap	/uimap	Índice dinâmico de arquivos (PowerShell) – evita buscas cegas
db	/db	Regras Supabase/Postgres + índice das tabelas atuais
ops	/ops	Deploy Railway, triage de logs, segurança de env vars
git	/git	Branching strategy + Conventional Commits cirúrgicos
status	/status	Snapshot rápido: git log + git status em 3 linhas

Meta-regra de autoevolução

Ao final de qualquer sessão, basta dizer **"update skills with what we did today"** para que o Claude identifique automaticamente quais arquivos, tabelas ou regras mudaram e reescreva os arquivos das skills correspondentes, mantendo-os concisos.

9. DevOps e Deploy

Workflow Git

- Branch main: production-ready apenas – nunca commit direto
- feat// para novas features
- fix// para bug fixes
- Commits cirúrgicos separados: feat(db):, feat(ui):, feat(api):

Railway

```
railway up → deplova branch atual  
railway logs --tail 20 → últimas 20 linhas de log
```

- Segredos gerenciados no dashboard Railway ou .env.local (gitignored)
- Nunca expor valores de variáveis de ambiente no chat

10. Padrões de Segurança

- RLS habilitado em todas as tabelas com dados de usuário
- Middleware Next.js valida sessão em todas as rotas protegidas
- Service-role key apenas no API server (nunca no cliente)

- Variáveis de ambiente nunca expostas em código-fonte ou chat
- CORS: restringir origens após fase de desenvolvimento
- Helmet: headers HTTP de segurança habilitados no Fastify
- Senhas: gerenciadas exclusivamente pelo Supabase Auth (sem hash manual)
- Importação de usuários: validação de campos antes de criar no Auth

CheckFlow – Documentação Técnica e Funcional

Gerado automaticamente em 05/06/2026

Este documento é gerado a partir do estado atual do repositório.